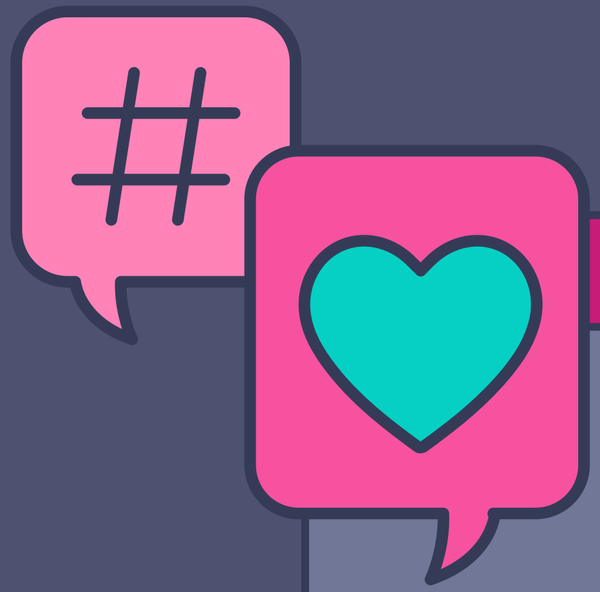


Paradigma
Funcional

Haskell

Indices

1 Paradigma Funcional

2 Haskell

3 Imutabilidade e Funções Pura

4 Data e Lista

5 Funções e Guards

6 Exercícios



O que é Programação Funcional (PF)?

A PF é um paradigma de programação que trata a computação como a avaliação de funções matemáticas.

Objetivo Principal: Reduzir efeitos colaterais e aumentar a previsibilidade e corretude do código.

Diferença da POO: Em vez de "objetos" com estado mutável, focamos em funções que transformam dados.

Fundamentos do Paradigma Funcional



Funções Puras: A saída depende apenas da entrada (argumentos). Não causam efeitos colaterais (modificar variáveis globais, I/O, etc.).

Imutabilidade: Variáveis, uma vez definidas, não podem mudar de valor. Isso simplifica o raciocínio sobre o fluxo de dados.

Expressões vs. Comandos: Tudo é uma expressão que retorna um valor, não um comando que altera o estado do programa.

Composição de Funções: Combinar funções menores para construir funções mais complexas.

Haskell: A Linguagem Funcional Pura



Linguagem Funcional Pura: Aderência estrita aos princípios funcionais (todas as funções são puras por padrão).

Fortemente Tipada: O tipo de todas as expressões é verificado em tempo de compilação, o que previne muitos erros.

Inferência de Tipos: Você não precisa declarar todos os tipos, o compilador (GHC) os infere para você.

Imutabilidade e Funções Puras em Ação

Imutabilidade: Em Haskell, você não tem x = x + 1. Uma vez que x é definido, ele mantém seu valor.

Função Pura: Uma função só depende de seus argumentos. Se você fornecer a mesma entrada, sempre obterá a mesma saída.

Tipos Personalizados: Data

- Propósito: Definir estruturas de dados imutáveis que representam conceitos do nosso domínio (Livraria). (como uma classe)
- Como funciona: data cria um novo tipo. Seus "campos" são chamados de Construtores de Dados. Os dados são encapsulados e não podem ser alterados após a criação.

Listas em Haskell

- Listas são o recipiente primário para coleções de dados de um mesmo tipo (ex: uma lista de Livro).
- Acessando o Primeiro Elemento (Padrões de Lista):
- Em vez de índices, usamos padrões de casamento (Pattern Matching):
- * [ ]: Lista vazia.
- * (x:_): Pega o primeiro item (x) e ignora o resto (_).

Funções de alta ordem

- Funções de Alta Ordem (High-Order Functions): Funções que recebem outras funções como argumento.
- map: Aplica uma função a todos os elementos de uma lista, retornando uma nova lista.
- sum: Reduz a lista de números a um único valor somando todos os elementos.

Condicionais Funcionais: Os Guards

- Propósito: Controlar o fluxo do programa, definindo diferentes resultados com base em condições, de forma pura.
- Como funciona: Eles substituem as estruturas if/else longas de linguagens imperativas, tornando o código mais legível e declarativo.

Ação IO: main

- Pureza vs. Efeitos Colaterais: Haskell separa estritamente as funções puras daquelas que causam efeitos colaterais (Input/Output – IO).
- main: É a porta de entrada para a execução e é a única parte que pode executar ações IO.
- * putStr / putStr / print: Funções de IO que permitem exibir dados ao usuário.

Questão 1

- No Café Leblanc, o dono Sojiro gerencia as bebidas vendidas no dia.
- Cada bebida tem um nome, um tipo (por exemplo, “Café”, “Chá” ou “Suco”) e um preço.
- Crie um sistema semelhante ao do exemplo da livraria, com os seguintes pontos:
- Defina um tipo Bebida com os campos nome, tipo e preco.
- Crie um tipo Pedido que contém uma lista de bebidas e um status (Aberto, Entregue, Cancelado).
- Implemente uma função valorTotalPedido que soma os preços das bebidas e adiciona uma taxa de serviço fixa de R$5,00 (exceto se o pedido estiver Cancelado, caso em que o valor deve ser 0).
- Faça uma função primeiraBebida :: Pedido -> String que retorna o nome da primeira bebida do pedido ou uma mensagem caso a lista esteja vazia.



Questão 2

- Nas lojas de Hyrule, Link pode comprar itens com diferentes valores.
- Cada item tem um nome, uma categoria (“Arma”, “Poção”, “Equipamento”) e um preço.
- Implemente um sistema seguindo os passos:
- Crie os tipos Item e CompraZelda, com campos equivalentes aos usados em Livro e Compra.
- Crie uma função calculaDesconto :: [Item] -> Double que retorna 10% de desconto se o total dos itens for acima de 200.
- Use guards em uma função valorFinal :: CompraZelda -> Double que aplique o desconto quando apropriado, e adicione um frete fixo de 15.0 quando o total for menor ou igual a 200.
- Faça alguns exemplos de compras e mostre no main o valor final e os itens da compra.



Questão 3

- Uma casa de shows registra as bandas que tocaram em um evento.
- Cada banda tem um nome, um gênero musical e um valor do cachê.
- Crie um sistema que permita calcular o custo total de um evento:
- Defina os tipos Banda e Evento (com lista de bandas e status do evento: Ativo, Encerrado, Cancelado).
- Implemente custoTotalEvento :: Evento -> Double, que: soma todos os cachês, adiciona uma taxa de produção de 20% sobre o total, e retorna 0.0 se o evento foi Cancelado.
- Crie uma função bandaAbertura:: Evento -> String que exibe o nome da banda que irá abrir a casa (ou mensagem se não houver nenhuma) e uma semelhante que exiba o nome da banda que tocará por último fechando a noite na casa. (as funções deve ser chamadas)
- Na main, crie 3 eventos diferentes (um ativo, um encerrado e um cancelado) e mostre o custo de cada um.



Questão 4

- Na Casa de Banhos do mundo espiritual (gerenciada pela bruxa Yubaba), várias criaturas pagam por diferentes tipos de serviço.
- Cada serviço tem um nome, um tipo (ex: “Banho”, “Massagem”, “Banquete”) e um preço base.
- Implemente um sistema de cálculo detalhado, com as seguintes regras:
- Crie os tipos Servico e Atendimento (com lista de serviços e status EmAndamento, Finalizado, Cancelado).
- Crie uma função bonusEspiritual :: [Servico] -> Double que: soma todos os preços dos serviços; aplica um bônus de 25% se houver mais de 3 serviços; aplica um desconto de 10% se o total ultrapassar 500.
- Implemente valorFinalAtendimento :: Atendimento -> Double usando guards para aplicar as regras e zerar o valor se o atendimento for cancelado.
- Adicione uma função descricaoPrimeiroServico :: Atendimento -> String que retorna o nome e tipo do primeiro serviço.
- No main, demonstre atendimentos variados e seus valores finais.



Fim da aula